In [1]:
```
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt
import random
import numpy as np
```

```
WARNING: All log messages before absl::InitializeLog() is called are writt
en to STDERR
I0000 00:00:1793420192.504234    8832 port.cc:153] oneDNN custom operation
s are on. You may see slightly different numerical results due to floatin
g-point round-off errors from different computation orders. To turn them o
ff, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
I0000 00:00:1793420192.508183    8832 cudart_stub.cc:31] Could not find cu
da drivers on your machine, GPU will not be used.
I0000 00:00:1793420192.917469    8832 cpu_feature_guard.cc:227] This Tenso
rFlow binary is optimized to use available CPU instructions in performanc
e-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operatio
ns, rebuild TensorFlow with the appropriate compiler flags.
/usr/lib/python3/dist-packages/scipy/__init__.py:146: UserWarning: A NumPy
version >=1.17.3 and <1.25.0 is required for this version of SciPy (detect
ed version 1.26.4
  warnings.warn(f"A NumPy version >={np_minversion} and <{np_maxversion}"
WARNING: All log messages before absl::InitializeLog() is called are writt
en to STDERR
I0000 00:00:1793420194.167982    8832 port.cc:153] oneDNN custom operation
s are on. You may see slightly different numerical results due to floatin
g-point round-off errors from different computation orders. To turn them o
ff, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
I0000 00:00:1793420194.169603    8832 cudart_stub.cc:31] Could not find cu
da drivers on your machine, GPU will not be used.
```

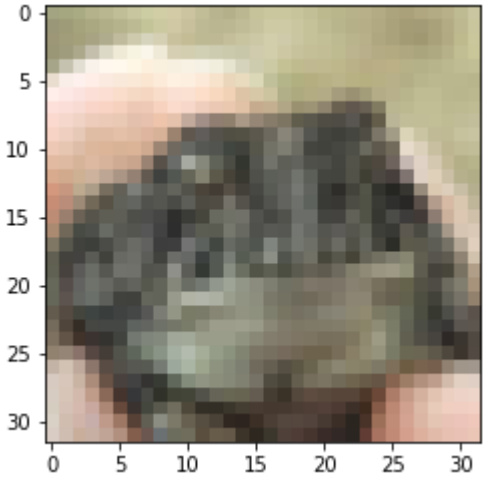
In [2]:
```
cifar10 = tf.keras.datasets.cifar10 # Importing cifar10 dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data() # Splitting it

for i in (x_train, y_train, x_test, y_test):
  print(i.shape)
```

```
(50000, 32, 32, 3)
(50000, 1)
(10000, 32, 32, 3)
(10000, 1)
```

In [3]:
```
plt.imshow(x_train[25])
print(y_train[25])
plt.show()
```

```
[6]
```



In [4]:
```
#scaling the values
x_train = x_train/255
x_test = x_test/255
# 0/255 = 255
#255/255 = 1 maximum value
```

In [5]:
```
print(x_train.shape, x_test.shape)
```

```
(50000, 32, 32, 3) (10000, 32, 32, 3)
```

In [11]:
```
num_classes = 10
class_names = ['airplane','automobile','bird','cat','deer',
               'dog','frog','horse','ship','truck']
```

In [12]:
```
# Convert class vectors to binary class matrices.
y_train = tf.keras.utils.to_categorical(y_train, num_classes)
y_test = tf.keras.utils.to_categorical(y_test, num_classes)
```

In [10]:
```
# Plotting the Model Accuracy & Model Loss vs Epochs
print(y_train.shape)
plt.figure(figsize=[10,8])

# summarize history for accuracy
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy', size=25, pad=20)
plt.ylabel('Accuracy', size=15)
plt.xlabel('Epoch', size=15)
plt.legend(['train', 'test'], loc='upper left')


# summarize history for loss
plt.subplot(1,2,2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss', size=25, pad=20)
plt.ylabel('Loss', size=15)
plt.xlabel('Epoch', size=15)
plt.legend(['train', 'test'], loc='upper left')
plt.show()y_test.shape)
```

```
  File "/tmp/ipykernel_8832/620187440.py", line 23
    plt.show()y_test.shape)
                          ^
SyntaxError: unmatched ')'
```

In [ ]:
```
from tensorflow import keras

# Create a sequential model
model = keras.Sequential()

# Convolutional layers
model.add(keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=
model.add(keras.layers.MaxPooling2D((2, 2)))
model.add(keras.layers.Conv2D(64, (3, 3), activation='relu'))
model.add(keras.layers.MaxPooling2D((2, 2)))
model.add(keras.layers.Conv2D(64, (3, 3), activation='relu'))

# Fully connected layers
model.add(keras.layers.Flatten())
model.add(keras.layers.Dense(64, activation='relu'))
model.add(keras.layers.Dense(10, activation='softmax'))  # 10 classes for

# Compile the model
model.compile(optimizer='sgd', loss='categorical_crossentropy', metrics=[

# Print the model summary to check the architecture
model.summary()
```

In [ ]:
```
# Training the model
history = model.fit(x_train, y_train,
              batch_size=30,
              epochs=10,
              validation_data=(x_test, y_test),
              shuffle=True)
```

In [ ]:
```
import seaborn as sns
sns.lineplot(model.history.history)
```

In [ ]:
```
# Plotting the Model Accuracy & Model Loss vs Epochs
plt.figure(figsize=[10,8])

# summarize history for accuracy
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy', size=25, pad=20)
plt.ylabel('Accuracy', size=15)
plt.xlabel('Epoch', size=15)
plt.legend(['train', 'test'], loc='upper left')


# summarize history for loss
plt.subplot(1,2,2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss', size=25, pad=20)
plt.ylabel('Loss', size=15)
plt.xlabel('Epoch', size=15)
```

```
plt.legend(['train', 'test'], loc='upper left')
plt.show()
```

In [ ]: